Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу

«Операционные системы»

Группа: М8О-209БВ-25

Студент: Алимов Р.Р.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 31.10.26

Москва, 2026

Постановка задачи

Вариант 2.

Родительский процесс создаёт дочерний процесс. Первой строкой пользователь вводит имя файла, которое передаётся дочерней программе. Далее пользователь вводит строки с произвольным количеством чисел типа float. Дочерний процесс вычисляет сумму чисел каждой строки и записывает результаты в указанный файл.

Межпроцессное взаимодействие организовано через именованный объект POSIX shared memory, отображённый в адресные пространства обоих процессов. Для синхронизации чтения и записи используются именованные POSIX-семафоры. Необходимо проверять ошибки системных вызовов.

Общий метод и алгоритм решения

Программа состоит из двух исполняемых файлов: parent.out и child.out. Родитель считывает имя выходного файла, формирует уникальные имена объектов по своему PID, создаёт разделяемую память вызовом shm_open(), задаёт размер через ftruncate() и отображает память с помощью mmap() и флага MAP_SHARED. Затем он создаёт два именованных семафора empty и full и дочерний процесс через fork(). Дочерняя программа запускается через execl(), получая имена объектов в аргументах.

После открытия файла и объектов IPC ребёнок сообщает о готовности вызовом sem_post(empty). Родитель читает стандартный ввод блоками до 1024 байт, ожидает разрешения sem_wait(empty), записывает блок в общую память и уведомляет ребёнка через sem_post(full). Ребёнок ожидает sem_wait(full), разбирает символы, преобразует числа функцией strtof() и записывает сумму при завершении каждой строки. После обработки блока ребёнок освобождает буфер через sem_post(empty).

Конец ввода передаётся значением shared->size = 0. Поле shared->error используется для уведомления родителя об ошибке. После завершения дочернего процесса родитель вызывает waitpid(), закрывает дескрипторы и удаляет именованные IPC-объекты. По сравнению с лабораторной работой №1 неименованный канал pipe() и перенаправление dup2() заменены общей памятью и явной синхронизацией семафорами.

Использованные системные вызовы и функции

- shm_open(name, oflag, mode) — создаёт или открывает именованный объект разделяемой памяти; возвращает файловый дескриптор.
- ftruncate(fd, length) — устанавливает размер объекта shared memory.
- mmap(addr, length, prot, flags, fd, offset) — отображает объект в виртуальное адресное пространство процесса; используется MAP_SHARED.
- munmap(addr, length) — снимает отображение из адресного пространства текущего процесса.
- shm_unlink(name) — удаляет имя объекта разделяемой памяти.
- sem_open(name, ...) — создаёт или открывает именованный POSIX-семафор; sem_close() закрывает его, sem_unlink() удаляет имя.
- sem_wait(sem) — ожидает разрешение семафора; sem_post(sem) — освобождает разрешение и может разбудить ожидающий процесс.
- fork() — создаёт дочерний процесс; execl() — заменяет его программой child.out.
- read(fd, buf, count) — считывает ввод из терминала; memcpy(dest, src, size) — копирует данные в общую память.
- open(path, flags, mode), write(fd, buf, count), close(fd) — открывают результирующий файл, записывают сумму и закрывают дескрипторы.
- waitpid(pid, &status, 0) — ожидает завершение конкретного ребёнка и позволяет проверить статус.

• strtof(str, &end) — преобразует строку в float; snprintf() — форматирует сумму для записи в файл.

Код программы

shared.h

```
#ifndef SHARED_H
#define SHARED_H

#define _POSIX_C_SOURCE 200809L
#define BUFFER_SIZE 1024

typedef struct {
    int size;
    int error;
    char buffer[BUFFER_SIZE];
} SharedData;

#endif
```

parent.c

```
#include "shared.h"
#include <fcntl.h>
#include <semaphore.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void) {
    char filename[BUFFER_SIZE];
    int pos = 0;
    char c;

    write(STDOUT_FILENO, "Enter filename: ", 16);
    while (pos < BUFFER_SIZE - 1) {
        ssize_t n = read(STDIN_FILENO, &c, 1);
        if (n == -1) {
            perror("read");
            return 1;
        }
        if (n == 0 || c == '\n') break;
        filename[pos++] = c;
    }
    filename[pos] = '\0';

    if (pos == 0) {
        fprintf(stderr, "Empty filename\n");
        return 1;
    }

    char shm_name[64], empty_name[64], full_name[64];
    long pid_number = (long)getpid();
    snprintf(shm_name, sizeof(shm_name), "/lab3_shm_%ld", pid_number);
    snprintf(empty_name, sizeof(empty_name), "/lab3_empty_%ld", pid_number);
    snprintf(full_name, sizeof(full_name), "/lab3_full_%ld", pid_number);

    int shm_fd = shm_open(shm_name, O_CREAT | O_EXCL | O_RDWR, 0600);
    if (shm_fd == -1) {
        perror("shm_open");
        return 1;
    }
    if (ftruncate(shm_fd, sizeof(SharedData)) == -1) {
        perror("ftruncate");
```

```
    close(shm_fd);
    shm_unlink(shm_name);
    return 1;
}

SharedData *shared = mmap(NULL, sizeof(SharedData),
                          PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);
if (shared == MAP_FAILED) {
    perror("mmap");
    close(shm_fd);
    shm_unlink(shm_name);
    return 1;
}
shared->size = 0;
shared->error = 0;

/* empty начинается с 0: ребенок сообщит, когда будет готов. */
sem_t *empty = sem_open(empty_name, O_CREAT | O_EXCL, 0600, 0);
if (empty == SEM_FAILED) {
    perror("sem_open empty");
    munmap(shared, sizeof(SharedData));
    close(shm_fd);
    shm_unlink(shm_name);
    return 1;
}
sem_t *full = sem_open(full_name, O_CREAT | O_EXCL, 0600, 0);
if (full == SEM_FAILED) {
    perror("sem_open full");
    sem_close(empty);
    sem_unlink(empty_name);
    munmap(shared, sizeof(SharedData));
    close(shm_fd);
    shm_unlink(shm_name);
    return 1;
}

pid_t pid = fork();
if (pid == -1) {
    perror("fork");
    sem_close(empty);
    sem_close(full);
    sem_unlink(empty_name);
    sem_unlink(full_name);
    munmap(shared, sizeof(SharedData));
    close(shm_fd);
    shm_unlink(shm_name);
    return 1;
}

if (pid == 0) {
    execl("./child.out", "child.out", filename,
          shm_name, empty_name, full_name, (char *)NULL);
    perror("execl");
    shared->error = 1;
    sem_post(empty);
    _exit(1);
}

int error = 0;
char buffer[BUFFER_SIZE];
ssize_t bytes_read;

while ((bytes_read = read(STDIN_FILENO, buffer, sizeof(buffer))) > 0) {
    if (sem_wait(empty) == -1) {
        perror("sem_wait");
        error = 1;
        break;
    }
    if (shared->error) {
```

```
            fprintf(stderr, "Child process error\n");
            error = 1;
            break;
        }
        memcpy(shared->buffer, buffer, (size_t)bytes_read);
        shared->size = (int)bytes_read;
        if (sem_post(full) == -1) {
            perror("sem_post");
            error = 1;
            break;
        }
    }

    if (bytes_read == -1) {
        perror("read");
        error = 1;
    }

    if (!error) {
        if (sem_wait(empty) == -1) {
            perror("sem_wait");
            error = 1;
        } else if (shared->error) {
            fprintf(stderr, "Child process error\n");
            error = 1;
        } else {
            shared->size = 0;  /* Конец ввода */
            if (sem_post(full) == -1) {
                perror("sem_post");
                error = 1;
            }
        }
    }

    if (error) kill(pid, SIGTERM);

    int status;
    if (waitpid(pid, &status, 0) == -1) {
        perror("waitpid");
        error = 1;
    } else if (!WIFEXITED(status) || WEXITSTATUS(status) != 0) {
        error = 1;
    }

    sem_close(empty);
    sem_close(full);
    sem_unlink(empty_name);
    sem_unlink(full_name);
    munmap(shared, sizeof(SharedData));
    close(shm_fd);
    shm_unlink(shm_name);

    return error ? 1 : 0;
}
```

child.c

```
#include "shared.h"
#include <fcntl.h>
#include <semaphore.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <unistd.h>

static int add_number(char *number, int *len, float *sum) {
    if (*len == 0) return 0;
    number[*len] = '\0';
```

```
    char *end;
    float value = strtof(number, &end);
    if (end == number || *end != '\0') return -1;
    *sum += value;
    *len = 0;
    return 0;
}

static int write_sum(int fd, float sum) {
    char output[64];
    int len = snprintf(output, sizeof(output), "%f\n", sum);
    if (len < 0 || len >= (int)sizeof(output)) return -1;
    return write(fd, output, (size_t)len) == len ? 0 : -1;
}

int main(int argc, char *argv[]) {
    if (argc != 5) {
        fprintf(stderr, "Wrong arguments\n");
        return 1;
    }

    int shm_fd = shm_open(argv[2], O_RDWR, 0);
    if (shm_fd == -1) {
        perror("shm_open");
        return 1;
    }
    SharedData *shared = mmap(NULL, sizeof(SharedData),
                              PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);
    if (shared == MAP_FAILED) {
        perror("mmap");
        close(shm_fd);
        return 1;
    }

    sem_t *empty = sem_open(argv[3], 0);
    sem_t *full = sem_open(argv[4], 0);
    if (empty == SEM_FAILED || full == SEM_FAILED) {
        perror("sem_open");
        if (empty != SEM_FAILED) sem_close(empty);
        if (full != SEM_FAILED) sem_close(full);
        munmap(shared, sizeof(SharedData));
        close(shm_fd);
        return 1;
    }

    int file_fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (file_fd == -1) {
        perror("open");
        shared->error = 1;
        sem_post(empty);
        sem_close(empty);
        sem_close(full);
        munmap(shared, sizeof(SharedData));
        close(shm_fd);
        return 1;
    }

    sem_post(empty);  /* Сообщаем родителю, что готовы */

    char number[BUFFER_SIZE + 1];
    int number_len = 0;
    int line_started = 0;
    float sum = 0.0f;
    int error = 0;

    while (1) {
        if (sem_wait(full) == -1) {
            perror("sem_wait");
            error = 1;
```

```
            break;
        }
        if (shared->size == 0) {
            sem_post(empty);
            break;
        }

        for (int i = 0; i < shared->size; i++) {
            char c = shared->buffer[i];
            if (c == ' ' || c == '\t' || c == '\n') {
                if (add_number(number, &number_len, &sum) == -1) {
                    fprintf(stderr, "Invalid number\n");
                    error = 1;
                    break;
                }
                if (c == '\n') {
                    if (write_sum(file_fd, sum) == -1) {
                        perror("write");
                        error = 1;
                        break;
                    }
                    sum = 0.0f;
                    line_started = 0;
                } else {
                    line_started = 1;
                }
            } else {
                if (number_len == BUFFER_SIZE) {
                    fprintf(stderr, "Number too long\n");
                    error = 1;
                    break;
                }
                number[number_len++] = c;
                line_started = 1;
            }
        }

        if (error) shared->error = 1;
        if (sem_post(empty) == -1) {
            perror("sem_post");
            error = 1;
        }
        if (error) break;
    }

    if (!error && add_number(number, &number_len, &sum) == -1) {
        fprintf(stderr, "Invalid number\n");
        error = 1;
    }
    if (!error && line_started && write_sum(file_fd, sum) == -1) {
        perror("write");
        error = 1;
    }

    close(file_fd);
    sem_close(empty);
    sem_close(full);
    munmap(shared, sizeof(SharedData));
    close(shm_fd);
    return error ? 1 : 0;
}
```

Makefile

```
all: parent.out child.out

parent.out: parent.c shared.h
    gcc -std=c11 -Wall -Wextra -pedantic -pthread -o parent.out parent.c -lrt
```

```
child.out: child.c shared.h
    gcc -std=c11 -Wall -Wextra -pedantic -pthread -o child.out child.c -lrt

clean:
    rm -f parent.out child.out

.PHONY: all clean
```

Протокол работы программы

Сборка выполнена командой make с флагами -std=c11 -Wall -Wextra -pedantic -pthread -lrt. Программы parent.out и child.out собраны без предупреждений. Ниже приведены результаты контрольных запусков подготовленной версии программы; ввод завершается EOF (Ctrl-D при интерактивной работе).

Тест 1. Дробные числа

```
$ ./parent.out
Enter filename: res.txt
2.222 1.456 345.03 23.055
0.11 0.29 0.5 00.03 0.07
000.000 111.110 222.223
^D
$ cat res.txt
371.763000
1.000000
333.333008
```

Суммы соответствуют введённым числам в пределах точности типа float. Значение 333.333008 связано с двоичным представлением дробных чисел.

Тест 2. Отрицательные и дробные числа

```
$ ./parent.out
Enter filename: res2.txt
1.5 2.5 10
4.2 0.8
100 -20.5
^D
$ cat res2.txt
14.000000
5.000000
79.500000
```

Тест 3. Строка длиннее одного блока

Для проверки передачи по нескольким блокам сформирована строка из 700 единиц, разделённых пробелами. Её длина превышает размер буфера 1024 байта.

```
$ { echo "res3.txt"; python3 -c 'print("1 " * 700)'; } | ./parent.out
$ cat res3.txt
700.000000
```

Тест 4. Некорректное число

```
$ ./parent.out
Enter filename: err.txt
234.3545 643.234 53.45
34 34 654.65
dfvkms
^D
Invalid number
Child process error
```

Дочерний процесс обнаруживает некорректный числовой токен через strtof(), сообщает об ошибке через общую память и завершает работу с ненулевым кодом. Родитель также завершает работу с ошибкой. Ранее успешно обработанные строки могут уже находиться в выходном файле.

Тест 5. Две одновременно работающие пары процессов

```
$ printf 'run_a.txt\n11 11.5\n' | ./parent.out &
$ printf 'run_b.txt\n22 22.5\n' | ./parent.out &
$ wait
$ cat run_a.txt
22.500000
$ cat run_b.txt
44.500000
```

Оба запуска успешно завершены. Имена объектов shared memory и семафоров формируются из PID родительского процесса, поэтому пары используют независимые IPC-объекты.

Вывод

В ходе лабораторной работы были получены практические навыки организации межпроцессного взаимодействия в Linux с использованием именованной разделяемой памяти POSIX и семафоров. На основе лабораторной работы №1 реализованы создание и отображение общей памяти через shm_open(), ftruncate() и mmap(), синхронизация передачи данных через sem_wait() и sem_post(), а также удаление IPC-объектов после завершения. Программа сохраняет функциональность варианта 2: дочерний процесс суммирует числа float по строкам и записывает результаты в файл. Проверены обычный ввод, большие строки, обработка ошибочных данных и независимая работа двух пар процессов.